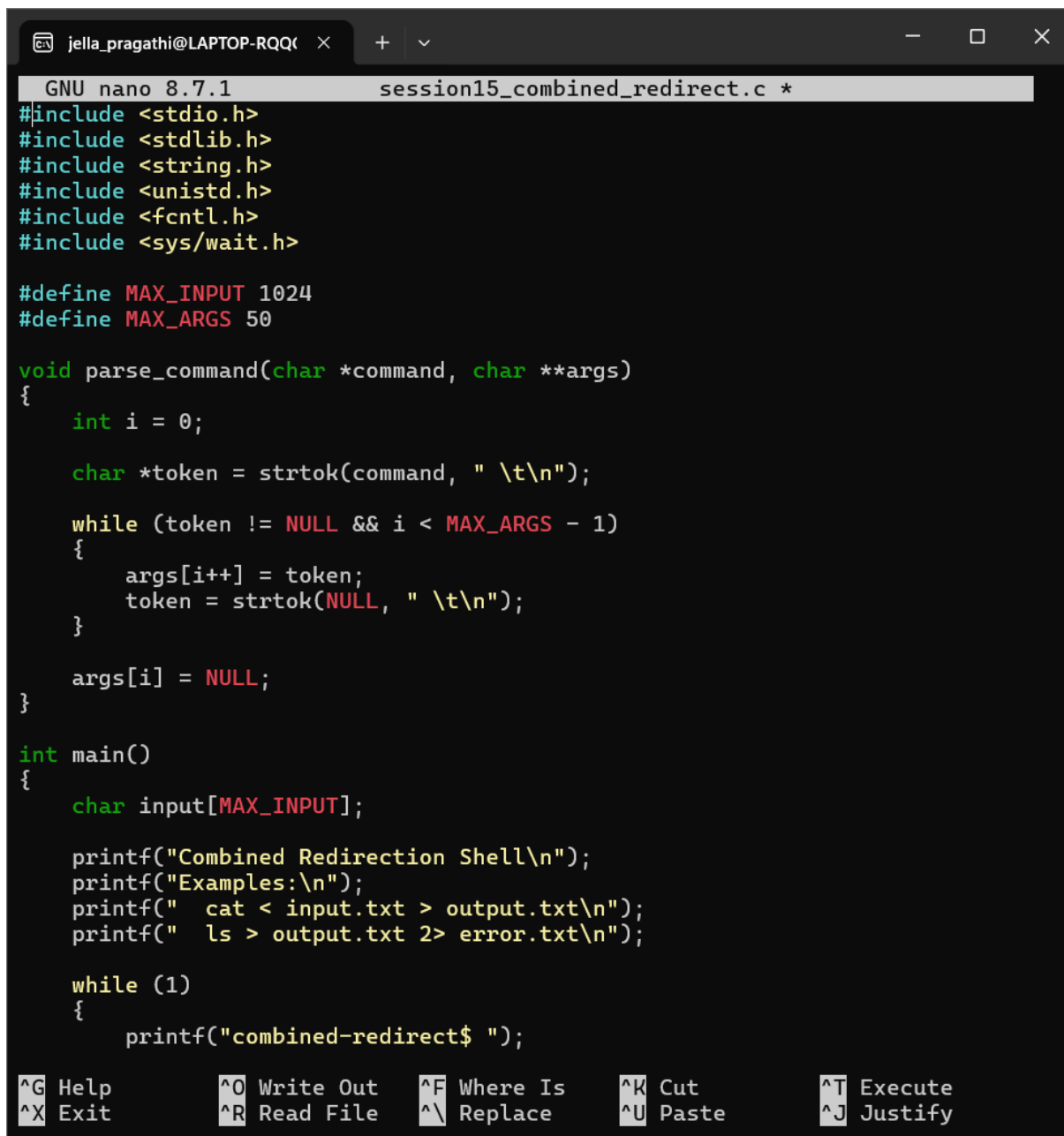


Skill Week 8

Session 15

Task 1

To Parse Combined Redirection, Duplicate File Descriptors, Merge Streams, Verify Ordering, Handle Edge Cases, Test Integrated Outputs.



```
GNU nano 8.7.1 session15_combined_redirect.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

#define MAX_INPUT 1024
#define MAX_ARGS 50

void parse_command(char *command, char **args)
{
    int i = 0;

    char *token = strtok(command, " \\t\\n");

    while (token != NULL && i < MAX_ARGS - 1)
    {
        args[i++] = token;
        token = strtok(NULL, " \\t\\n");
    }

    args[i] = NULL;
}

int main()
{
    char input[MAX_INPUT];

    printf("Combined Redirection Shell\\n");
    printf("Examples:\\n");
    printf("  cat < input.txt > output.txt\\n");
    printf("  ls > output.txt 2> error.txt\\n");

    while (1)
    {
        printf("combined-redirect$ ");
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute
^X Exit	^R Read File	^\\ Replace	^U Paste	^J Justify

```
jella_pragathi@LAPTOP-RQQ( x + v - □ ×
GNU nano 8.7.1 session15_combined_redirect.c *
fflush(stdout);

if (fgets(input, sizeof(input), stdin) == NULL)
    break;

input[strcspn(input, "\n")] = '\0';

if (strcmp(input, "exit") == 0)
    break;

char command[MAX_INPUT];
strcpy(command, input);

char input_file[MAX_INPUT] = "";
char output_file[MAX_INPUT] = "";
char error_file[MAX_INPUT] = "";

/*
 * Find input redirection <
 */
char *in_redirect = strchr(command, '<');

if (in_redirect != NULL)
{
    char *start = in_redirect + 1;

    while (*start == ' ' || *start == '\t')
        start++;

    char *end = start;

    while (*end != '\0' &&
           *end != '>' &&
           !(end[0] == '2' && end[1] == '>'))
    {
        end++;
    }
}
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute
^X Exit	^R Read File	^_\ Replace	^U Paste	^J Justify

GNU nano 8.7.1 session15_combined_redirect.c *

```
int length = end - start;

if (length <= 0)
{
    printf("Error: No input file specified.\n");
    continue;
}

strncpy(input_file, start, length);
input_file[length] = '\0';

char *space = strpbrk(input_file, " \t");

if (space != NULL)
    *space = '\0';

/*
 * Remove < input.txt from command
 */
memmove(in_redirect, end, strlen(end) + 1);
}

/*
 * Find error redirection 2>
 */
char *err_redirect = strstr(command, "2>");

if (err_redirect != NULL)
{
    char *start = err_redirect + 2;

    while (*start == ' ' || *start == '\t')
        start++;

    char *end = start;

    while (*end != '\0' &&
```

^G Help
^X Exit

^O Write Out
^R Read File

^F Where Is
^_ Replace

^K Cut
^U Paste

^T Execute
^J Justify

GNU nano 8.7.1 session15_combined_redirect.c *

```
    while (*end != '\\0' &&
           *end != ' ' &&
           *end != '\\t')
    {
        end++;
    }

    int length = end - start;

    if (length <= 0)
    {
        printf("Error: No error file specified.\\n");
        continue;
    }

    strncpy(error_file, start, length);
    error_file[length] = '\\0';

    /*
     * Remove 2> error.txt from command
     */
    *err_redirect = '\\0';
}

/*
 * Find output redirection >
 */
char *out_redirect = strchr(command, '>');

if (out_redirect != NULL)
{
    char *start = out_redirect + 1;

    while (*start == ' ' || *start == '\\t')
        start++;

    char *end = start;
```

^G Help
^X Exit

^O Write Out
^R Read File

^F Where Is
^ Replace

^K Cut
^U Paste

^T Execute
^J Justify

```
jella_pragathi@LAPTOP-RQQK: ~$ nano session15_combined_redirect.c
GNU nano 8.7.1 session15_combined_redirect.c *
    while (*end != '\0' &&
           *end != ' ' &&
           *end != '\t')
    {
        end++;
    }

    int length = end - start;

    if (length <= 0)
    {
        printf("Error: No output file specified.\n");
        continue;
    }

    strncpy(output_file, start, length);
    output_file[length] = '\0';

    /*
     * Remove > output.txt from command
     */
    *out_redirect = '\0';
}

/*
 * Remove trailing spaces
 */
int len = strlen(command);

while (len > 0 &&
       (command[len - 1] == ' ' ||
        command[len - 1] == '\t'))
{
    command[len - 1] = '\0';
    len--;
}

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify
```

```
jella_pragathi@LAPTOP-RQQ( × + ▾
GNU nano 8.7.1 session15_combined_redirect.c *

char *args[MAX_ARGS];

parse_command(command, args);

if (args[0] == NULL)
{
    printf("Error: No command specified.\n");
    continue;
}

/*
 * Display execution plan
 */
printf("\nExecution plan:\n");
printf("Command: %s\n", args[0]);

if (input_file[0] != '\0')
    printf("Input: %s\n", input_file);

if (output_file[0] != '\0')
    printf("Output: %s\n", output_file);

if (error_file[0] != '\0')
    printf("Error: %s\n", error_file);

printf("\n");

pid_t pid = fork();

if (pid < 0)
{
    perror("fork");
    continue;
}

if (pid == 0)
```

Help	Write Out	Where Is	Cut	Execute
Exit	Read File	Replace	Paste	Justify

```
jella_pragathi@LAPTOP-RQQ( x + v
GNU nano 8.7.1 session15_combined_redirect.c *
/* Input redirection
*/
if (input_file[0] != '\0')
{
    int fd_in = open(input_file, O_RDONLY);

    if (fd_in < 0)
    {
        perror("open input");
        exit(EXIT_FAILURE);
    }

    if (dup2(fd_in, STDIN_FILENO) < 0)
    {
        perror("dup2 input");
        close(fd_in);
        exit(EXIT_FAILURE);
    }

    close(fd_in);
}

/*
* Output redirection
*/
if (output_file[0] != '\0')
{
    int fd_out = open(
        output_file,
        O_WRONLY | O_CREAT | O_TRUNC,
        0644
    );

    if (fd_out < 0)
    {
        perror("open output");
        exit(EXIT_FAILURE);
    }
}

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify
```

```
    }

    if (dup2(fd_out, STDOUT_FILENO) < 0)
    {
        perror("dup2 output");
        close(fd_out);
        exit(EXIT_FAILURE);
    }

    close(fd_out);
}

/*
 * Error redirection
 */
if (error_file[0] != '\0')
{
    int fd_err = open(
        error_file,
        O_WRONLY | O_CREAT | O_TRUNC,
        0644
    );

    if (fd_err < 0)
    {
        perror("open error");
        exit(EXIT_FAILURE);
    }

    if (dup2(fd_err, STDERR_FILENO) < 0)
    {
        perror("dup2 error");
        close(fd_err);
        exit(EXIT_FAILURE);
    }

    close(fd_err);
}
```

^G Help
^X Exit

^O Write Out
^R Read File

^F Where Is
^_ Replace

^K Cut
^U Paste

^T Execute
^J Justify

```

        execvp(args[0], args);

        perror("execvp");
        exit(EXIT_FAILURE);
    }
    else
    {
        waitpid(pid, NULL, 0);
    }
}

printf("Combined redirection shell terminated.\n");

return 0;
}
|

```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute
^X Exit	^R Read File	^\\ Replace	^U Paste	^J Justify

```

jella_pragathi@LAPTOP-RQQGT555:~$ cd ~/OSSP
nano session15_combined_redirect.c
jella_pragathi@LAPTOP-RQQGT555:~/OSSP$ gcc -Wall -Wextra -g session15_combined_redirect.c -o session15_combined_redirect
jella_pragathi@LAPTOP-RQQGT555:~/OSSP$ echo "This is Session 15 input redirection test." > input15.txt
jella_pragathi@LAPTOP-RQQGT555:~/OSSP$ ./session15_combined_redirect
Combined Redirection Shell
Examples:
    cat < input.txt > output.txt
    ls > output.txt 2> error.txt
combined-redirect$ cat < input15.txt > output15.txt

Execution plan:
Command: cat
Input: input15.txt
Output: output15.txt

combined-redirect$ exit
Combined redirection shell terminated.
jella_pragathi@LAPTOP-RQQGT555:~/OSSP$ cat output15.txt
This is Session 15 input redirection test.
jella_pragathi@LAPTOP-RQQGT555:~/OSSP$ ls -l output15.txt
-rw-r--r-- 1 jella_pragathi jella_pragathi 43 Sep 17 07:50 output15.txt
jella_pragathi@LAPTOP-RQQGT555:~/OSSP$ |

```

Task 2

To Parse Complex Commands, Combine Pipes and Redirections, Resolve Execution Order, Validate Syntax, Generate Execution Plans, Test Scenarios.



```
GNU nano 8.7.1 session15_complex.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

#define MAX_INPUT 1024
#define MAX_COMMANDS 10
#define MAX_ARGS 50

void parse_args(char *command, char **args)
{
    int i = 0;

    char *token = strtok(command, " \\t\\n");

    while (token != NULL && i < MAX_ARGS - 1)
    {
        args[i++] = token;
        token = strtok(NULL, " \\t\\n");
    }

    args[i] = NULL;
}

int main()
{
    char input[MAX_INPUT];

    printf("Complex Pipeline and Redirection Shell\\n");
    printf("Example:\\n");
    printf("  cat < input.txt | grep Session > result.txt\\n");

    while (1)
    {
        printf("complex-shell$ ");
```

Help	Write Out	Where Is	Cut	Execute
Exit	Read File	Replace	Paste	Justify

```
| fflush(stdout);

if (fgetc(input, sizeof(input), stdin) == NULL)
    break;

input[strcspn(input, "\n")] = '\0';

if (strcmp(input, "exit") == 0)
    break;

if (strlen(input) == 0)
    continue;

/*
 * Split command using pipe |
 */
char *commands[MAX_COMMANDS];
int command_count = 0;

char *token = strtok(input, "|");

while (token != NULL && command_count < MAX_COMMANDS)
{
    commands[command_count++] = token;
    token = strtok(NULL, "|");
}

printf("\nExecution Plan\n");
printf("-----\n");

for (int i = 0; i < command_count; i++)
{
    printf("Command %d: %s\n", i + 1, commands[i]);
}

printf("-----\n");
```

```
jella_pragathi@LAPTOP-RQQQ × + - □ ×
GNU nano 8.7.1 session15_complex.c *
int pipes[MAX_COMMANDS - 1][2];

/*
 * Create pipes
 */
for (int i = 0; i < command_count - 1; i++)
{
    if (pipe(pipes[i]) < 0)
    {
        perror("pipe");
        exit(EXIT_FAILURE);
    }
}

pid_t pids[MAX_COMMANDS];

/*
 * Create processes
 */
for (int i = 0; i < command_count; i++)
{
    pids[i] = fork();

    if (pids[i] < 0)
    {
        perror("fork");
        exit(EXIT_FAILURE);
    }

    if (pids[i] == 0)
    {
        char command[MAX_INPUT];
        strcpy(command, commands[i]);

        char *input_file = NULL;
        char *output_file = NULL;
    }
}

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify
```

```
/*
 * Input redirection <
 */
char *in_redirect = strchr(command, '<');

if (in_redirect != NULL)
{
    *in_redirect = '\0';

    char *file = strtok(in_redirect + 1, " \t\n");

    if (file != NULL)
        input_file = file;
}

/*
 * Output redirection >
 */
char *out_redirect = strchr(command, '>');

if (out_redirect != NULL)
{
    *out_redirect = '\0';

    char *file = strtok(out_redirect + 1, " \t\n");

    if (file != NULL)
        output_file = file;
}

char *args[MAX_ARGS];

parse_args(command, args);

if (args[0] == NULL)
{
```

^G Help
^X Exit

^O Write Out
^R Read File

^F Where Is
^_ Replace

^K Cut
^U Paste

^T Execute
^J Justify

```
jella_pragathi@LAPTOP-RQQ( × + - □ ×
GNU nano 8.7.1 session15_complex.c *
    fprintf(stderr, "Error: Empty command.\n");
    exit(EXIT_FAILURE);
}

/*
 * Connect input from previous pipe
 */
if (i > 0)
{
    if (dup2(pipes[i - 1][0], STDIN_FILENO) < 0)
    {
        perror("dup2 input pipe");
        exit(EXIT_FAILURE);
    }
}

/*
 * Connect output to next pipe
 */
if (i < command_count - 1)
{
    if (dup2(pipes[i][1], STDOUT_FILENO) < 0)
    {
        perror("dup2 output pipe");
        exit(EXIT_FAILURE);
    }
}

/*
 * File input redirection
 * This takes priority over pipe input.
 */
if (input_file != NULL)
{
    int fd = open(input_file, O_RDONLY);

    if (fd < 0)
```

G Help	^O Write Out	^F Where Is	^K Cut	^T Execute
X Exit	^R Read File	^_ Replace	^U Paste	^J Justify

```
jella_pragathi@LAPTOP-RQQ... × + ▾
GNU nano 8.7.1 session15_complex.c *
    perror("open input");
    exit(EXIT_FAILURE);
}

    if (dup2(fd, STDIN_FILENO) < 0)
    {
        perror("dup2 input");
        close(fd);
        exit(EXIT_FAILURE);
    }

    close(fd);
}

/*
 * File output redirection
 * This takes priority over pipe output.
 */
if (output_file != NULL)
{
    int fd = open(
        output_file,
        O_WRONLY | O_CREAT | O_TRUNC,
        0644
    );

    if (fd < 0)
    {
        perror("open output");
        exit(EXIT_FAILURE);
    }

    if (dup2(fd, STDOUT_FILENO) < 0)
    {
        perror("dup2 output");
        close(fd);
        exit(EXIT_FAILURE);
    }
}

|
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute
^X Exit	^R Read File	^_\ Replace	^U Paste	^J Justify

```
jella_pragathi@LAPTOP-RQQ( x + v - □ ×
GNU nano 8.7.1 session15_complex.c *
        exit(EXIT_FAILURE);
    }

    close(fd);
}

/*
 * Close all pipe descriptors
 */
for (int j = 0; j < command_count - 1; j++)
{
    close(pipes[j][0]);
    close(pipes[j][1]);
}

execvp(args[0], args);

perror("execvp");
exit(EXIT_FAILURE);
}
}

/*
 * Parent closes all pipe descriptors
 */
for (int i = 0; i < command_count - 1; i++)
{
    close(pipes[i][0]);
    close(pipes[i][1]);
}

/*
 * Wait for all children
 */
for (int i = 0; i < command_count; i++)
{
    waitpid(pids[i], NULL, 0);
}

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify
```

```
        printf("\nAll pipeline processes completed.\n\n");
    }

    printf("Complex shell terminated.\n");

    return 0;
}

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify
```

```

jella_pragathi@LAPTOP-RQQT555:~/OSSP$ cd ~/OSSP
nano session15_complex.c
jella_pragathi@LAPTOP-RQQT555:~/OSSP$ gcc -Wall -Wextra -g session15_complex.c -o session15_complex
jella_pragathi@LAPTOP-RQQT555:~/OSSP$ echo "Session 15 is about Linux system programming." > complex_input.txt
jella_pragathi@LAPTOP-RQQT555:~/OSSP$ cat complex_input.txt
Session 15 is about Linux system programming.
jella_pragathi@LAPTOP-RQQT555:~/OSSP$ ./session15_complex
Complex Pipeline and Redirection Shell
Example:
    cat < input.txt | grep Session > result.txt
complex-shell$ cat < complex_input.txt | grep Session > complex_output.txt

Execution Plan
-----
Command 1: cat < complex_input.txt
Command 2: grep Session > complex_output.txt
-----

All pipeline processes completed.

complex-shell$ exit
Complex shell terminated.
jella_pragathi@LAPTOP-RQQT555:~/OSSP$ cat complex_output.txt
Session 15 is about Linux system programming.
jella_pragathi@LAPTOP-RQQT555:~/OSSP$ |

```

Skill Session 16

Task 1

To Detect Background Jobs, Launch Non-Blocking Processes, Return Prompt Immediately, Store Job Information, Monitor Execution, Test Background Tasks.



```
GNU nano 8.7.1 session16_background.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#include <errno.h>

#define MAX_INPUT 1024
#define MAX_ARGS 50

void parse_command(char *command, char **args)
{
    int i = 0;

    char *token = strtok(command, " \t\n");

    while (token != NULL && i < MAX_ARGS - 1)
    {
        args[i++] = token;
        token = strtok(NULL, " \t\n");
    }

    args[i] = NULL;
}

int main()
{
    char input[MAX_INPUT];

    printf("Background Job Shell\n");
    printf("Example: sleep 10 &\n");

    while (1)
    {
        /*
         * Check for completed background processes
         */
    }
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute
^X Exit	^R Read File	^_\ Replace	^U Paste	^J Justify

jella_pragathi@LAPTOP-RQQ(× + ▾

GNU nano 8.7.1session16_background.c *

```
int status;
pid_t finished_pid;

while ((finished_pid = waitpid(-1, &status, WNOHANG)) > 0)
{
    printf("\n[Background job completed] PID: %d\n",
        finished_pid);
}

printf("job-shell$ ");
fflush(stdout);

if (fgets(input, sizeof(input), stdin) == NULL)
    break;

input[strcspn(input, "\n")] = '\0';

if (strcmp(input, "exit") == 0)
    break;

if (strlen(input) == 0)
    continue;

/*
 * Check whether command ends with &
 */
int background = 0;

int len = strlen(input);

while (len > 0 &&
        (input[len - 1] == ' ' ||
         input[len - 1] == '\t'))
{
    input[len - 1] = '\0';
    len--;
}
|
```

^G Help

^X Exit

^O Write Out

^R Read File

^F Where Is

^_ Replace

^K Cut

^U Paste

^T Execute

^J Justify

```
jella_pragathi@LAPTOP-RQQ... × + ▾
GNU nano 8.7.1 session16_background.c *
|  if (len > 0 && input[len - 1] == '&')
|  {
|      background = 1;
|      input[len - 1] = '\0';
|
|      while (len > 0 &&
|             (input[len - 1] == ' ' ||
|              input[len - 1] == '\t'))
|      {
|          input[len - 1] = '\0';
|          len--;
|      }
|  }
|
|  char command[MAX_INPUT];
|  strcpy(command, input);
|
|  char *args[MAX_ARGS];
|
|  parse_command(command, args);
|
|  if (args[0] == NULL)
|  {
|      printf("Error: No command specified.\n");
|      continue;
|  }
|
|  pid_t pid = fork();
|
|  if (pid < 0)
|  {
|      perror("fork");
|      continue;
|  }
|
|  if (pid == 0)
|  {
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute
^X Exit	^R Read File	^_\ Replace	^U Paste	^J Justify

```

        if (pid == 0)
        {
            execvp(args[0], args);

            perror("execvp");
            exit(EXIT_FAILURE);
        }

        if (background)
        {
            printf("[Background job started] PID: %d\n", pid);
        }
        else
        {
            waitpid(pid, NULL, 0);
            printf("[Foreground job completed] PID: %d\n", pid);
        }
    }

    /*
     * Clean up remaining child processes
     */
    while (waitpid(-1, NULL, 0) > 0)
    {
        /* Wait for remaining children */
    }

    printf("Background job shell terminated.\n");

    return 0;
}

```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute
 ^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify

```

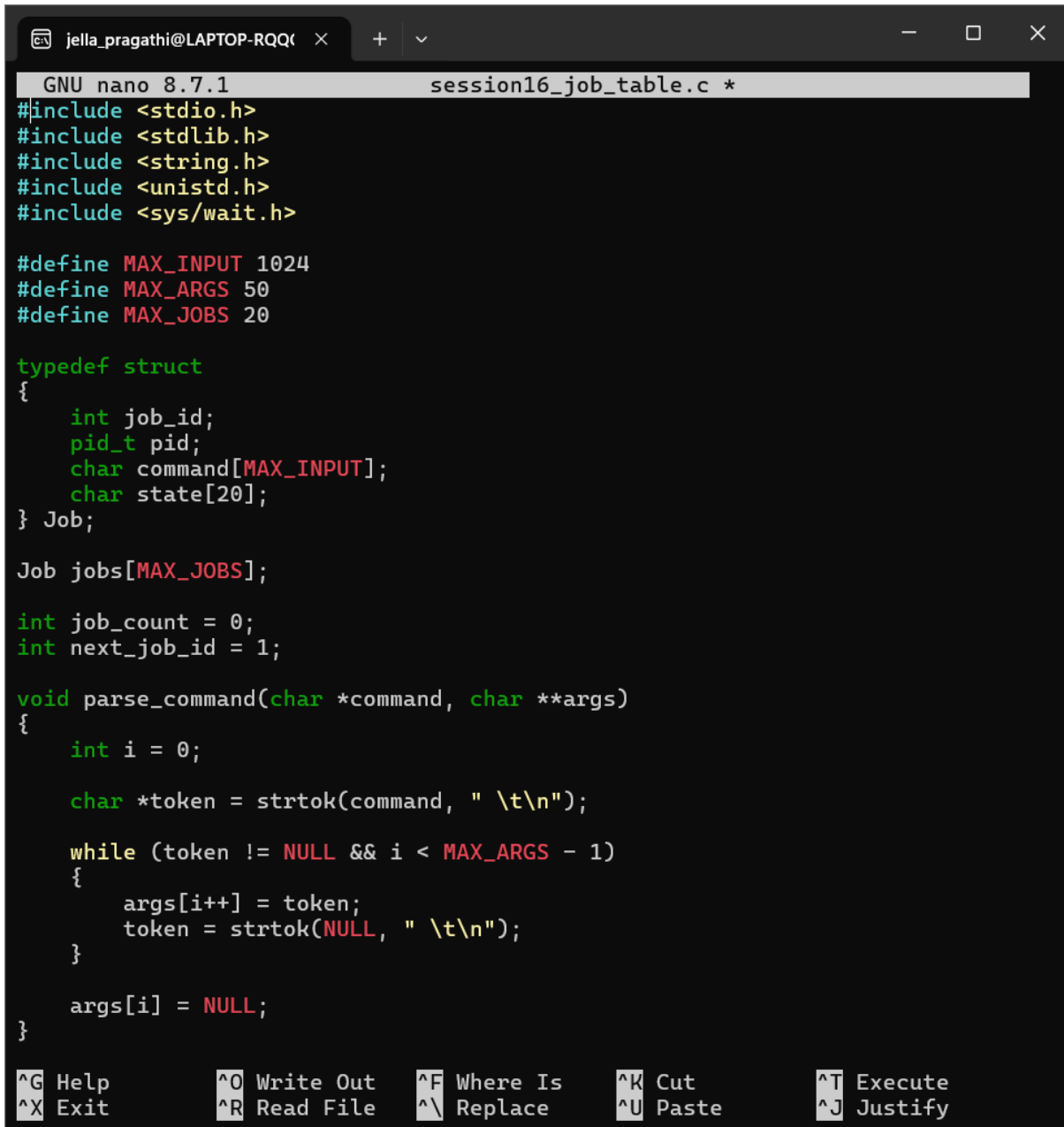
jella_pragathi@LAPTOP-RQQGT555: ~$ cd ~/OSSP
jella_pragathi@LAPTOP-RQQGT555: ~/OSSP$ nano session16_background.c
jella_pragathi@LAPTOP-RQQGT555: ~/OSSP$ gcc -Wall -Wextra -g session16_backgr
ound.c -o session16_background
jella_pragathi@LAPTOP-RQQGT555: ~/OSSP$ ./session16_background
Background Job Shell
Example: sleep 10 &
job-shell$ echo Hello
Hello
[Foreground job completed] PID: 1106
job-shell$ sleep 5 &
[Background job started] PID: 1131
job-shell$ echo Background test
Background test
[Foreground job completed] PID: 1134

[Background job completed] PID: 1131
job-shell$ exit
Background job shell terminated.
jella_pragathi@LAPTOP-RQQGT555: ~/OSSP$

```

Task 2

To Create Job Table, Store Process Groups, Track States, Update Status, Remove Completed Jobs, Validate Job Records.



```
GNU nano 8.7.1 session16_job_table.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_INPUT 1024
#define MAX_ARGS 50
#define MAX_JOBS 20

typedef struct
{
    int job_id;
    pid_t pid;
    char command[MAX_INPUT];
    char state[20];
} Job;

Job jobs[MAX_JOBS];

int job_count = 0;
int next_job_id = 1;

void parse_command(char *command, char **args)
{
    int i = 0;

    char *token = strtok(command, " \\t\\n");

    while (token != NULL && i < MAX_ARGS - 1)
    {
        args[i++] = token;
        token = strtok(NULL, " \\t\\n");
    }

    args[i] = NULL;
}

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute
^X Exit      ^R Read File ^\\ Replace   ^U Paste     ^J Justify
```

```
jella_pragathi@LAPTOP-RQQ: ~$ nano session16_job_table.c
GNU nano 8.7.1 session16_job_table.c *

void add_job(pid_t pid, char *command)
{
    if (job_count >= MAX_JOBS)
    {
        printf("Job table is full.\n");
        return;
    }

    jobs[job_count].job_id = next_job_id++;
    jobs[job_count].pid = pid;

    strcpy(jobs[job_count].command, command);
    strcpy(jobs[job_count].state, "RUNNING");

    printf("[%d] PID %d started: %s\n",
           jobs[job_count].job_id,
           jobs[job_count].pid,
           jobs[job_count].command);

    job_count++;
}

void update_jobs()
{
    int status;

    for (int i = 0; i < job_count; i++)
    {
        pid_t result = waitpid(
            jobs[i].pid,
            &status,
            WNOHANG
        );

        if (result > 0)
        {
            |
        }
    }
}

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify
```

```
jella_pragathi@LAPTOP-RQQC x + v
GNU nano 8.7.1 session16_job_table.c *
        strcpy(jobs[i].state, "DONE");
    }
}

void show_jobs()
{
    update_jobs();

    if (job_count == 0)
    {
        printf("No active jobs.\n");
        return;
    }

    printf("\nJob Table\n");
    printf("-----\n");
    printf("Job ID\tPID\tState\t\tCommand\n");
    printf("-----\n");

    for (int i = 0; i < job_count; i++)
    {
        printf("%d\t%d\t%s\t\t%s\n",
                jobs[i].job_id,
                jobs[i].pid,
                jobs[i].state,
                jobs[i].command);
    }

    printf("-----\n");
}

void remove_completed_jobs()
{
    int i = 0;

    while (i < job_count)
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute
^X Exit	^R Read File	^I Replace	^U Paste	^J Justify

```
jella_pragathi@LAPTOP-RQQ( x + v - □ ×
GNU nano 8.7.1 session16_job_table.c *
| {
    if (strcmp(jobs[i].state, "DONE") == 0)
    {
        printf("Removing completed job [%d] PID %d\n",
               jobs[i].job_id,
               jobs[i].pid);

        for (int j = i; j < job_count - 1; j++)
        {
            jobs[j] = jobs[j + 1];
        }

        job_count--;
    }
    else
    {
        i++;
    }
}

int main()
{
    char input[MAX_INPUT];

    printf("Job Table Shell\n");
    printf("Commands:\n");
    printf("  command &    -> background job\n");
    printf("  jobs          -> show job table\n");
    printf("  cleanup       -> remove completed jobs\n");
    printf("  exit          -> quit\n");

    while (1)
    {
        update_jobs();

        printf("job-table$ ");
    }
}

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify
```

```
jella_pragathi@LAPTOP-RQQQ( x + v
GNU nano 8.7.1 session16_job_table.c *
    fflush(stdout);

    if (fgets(input, sizeof(input), stdin) == NULL)
        break;

    input[strcspn(input, "\n")] = '\0';

    if (strcmp(input, "exit") == 0)
        break;

    if (strcmp(input, "jobs") == 0)
    {
        show_jobs();
        continue;
    }

    if (strcmp(input, "cleanup") == 0)
    {
        remove_completed_jobs();
        continue;
    }

    if (strlen(input) == 0)
        continue;

    int background = 0;

    int len = strlen(input);

    while (len > 0 &&
           (input[len - 1] == ' ' ||
            input[len - 1] == '\t'))
    {
        input[len - 1] = '\0';
        len--;
    }

|
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute
^X Exit	^R Read File	^_ Replace	^U Paste	^J Justify

```
if (len > 0 && input[len - 1] == '&')
{
    background = 1;

    input[len - 1] = '\\0';

    len--;

    while (len > 0 &&
           (input[len - 1] == ' ' ||
            input[len - 1] == '\\t'))
    {
        input[len - 1] = '\\0';
        len--;
    }
}

char command[MAX_INPUT];
strcpy(command, input);

char command_for_exec[MAX_INPUT];
strcpy(command_for_exec, input);

char *args[MAX_ARGS];

parse_command(command_for_exec, args);

if (args[0] == NULL)
{
    printf("Error: No command specified.\\n");
    continue;
}

| pid_t pid = fork();
```

^G Help
^X Exit

^O Write Out
^R Read File

^F Where Is
^\\ Replace

^K Cut
^U Paste

^T Execute
^J Justify


```
jella_pragathi@LAPTOP-RQQG: ~$ cd ~/OSSP
jella_pragathi@LAPTOP-RQQG: ~/OSSP$ nano session16_job_table.c
jella_pragathi@LAPTOP-RQQG: ~/OSSP$ gcc -Wall -Wextra -g session16_job_table.c -o session16_job_table
jella_pragathi@LAPTOP-RQQG: ~/OSSP$ ./session16_job_table
Job Table Shell
Commands:
  command &    -> background job
  jobs         -> show job table
  cleanup      -> remove completed jobs
  exit         -> quit
job-table$ sleep 10 &
[1] PID 1208 started: sleep 10
job-table$ jobs

Job Table
-----
Job ID  PID    State    Command
-----
1       1208    RUNNING  sleep 10
-----
job-table$ jobs

Job Table
-----
Job ID  PID    State    Command
-----
1       1208    DONE     sleep 10
-----
```

```
job-table$ cleanup
Removing completed job [1] PID 1240
job-table$ |
```